KONVERT NOTIFY ME & WISHLIST

Technical Audit Report

Read-only repository audit | 12 October 2026 | Source: current repository files

No app files, databases, Shopify stores, or production systems were modified or queried. Environment-variable values are omitted. The V1 PRD and A1–A12 acceptance criteria were unavailable; PRD compliance is NOT VERIFIED.

1. Executive summary

The repository contains a React Router Shopify app, product-page Theme App Extension, SQLite Shopify session storage, and separate MySQL application schema. Implemented code includes merchant reports, Notify Me signup, authenticated Wishlist and My Wishlist, paid-order ingestion/matching, feature settings, and one-shot availability polling.

Material gaps: provider adapter is a stub; available worker does not process HandoffJobs; no scheduler is configured; GDPR deletion/retention and opt-out flows were not found. Live MySQL state, Shopify Admin setup, provider, scheduling and test outcomes are NOT VERIFIED. Docs are stale in places.

2. Verified architecture

```
flowchart LR
Merchant[Shopify Admin] --> App[Embedded React Router app]
Shopper[Storefront] --> Blocks[Notify and Wishlist blocks]
Blocks --> Proxy[Shopify App Proxy] --> App
App --> Session[(SQLite sessions)]
App --> MySQL[(MySQL app data)]
App --> AdminAPI[Shopify Admin GraphQL]
Webhooks[Shopify webhooks] --> App
Worker[One-shot worker] --> AdminAPI
Worker --> MySQL
MySQL --> Reports[Merchant reporting]
MySQL -. no configured runner .-> Handoff[Handoff jobs/provider stub]
sequenceDiagram
Shopper->>Block: Submit Notify / Wishlist
Block->>Proxy: Signed request
Proxy->>App: Shop/customer context
App->>AdminAPI: Verify Shopify records
AdminAPI-->>App: Verified data
App->>MySQL: Save request/state/event
Worker->>AdminAPI: Poll availability
Worker->>MySQL: Record transition/jobs/matches
```

Implemented in code: admin auth, proxy routes, GraphQL, webhook handlers, two Prisma clients, services and UI. External setup not established: active app linkage/scopes/proxy/webhooks, MySQL state, scheduler/provider and protected-customer-data access.

3. Repository and dependencies

Root: C:\Users\ashis\OneDrive\Desktop\konvert-notify-wishlist. app/routes: pages/API/auth/webhooks; app/services: domain logic; app/workers and scripts: one-shot worker; extensions/konvert-notify-theme: Liquid, JS, CSS, locales; prisma: separate SQLite/MySQL schemas and migrations; docs: setup/architecture; public: static assets. .shopify, .react-router, build and node_modules are generated/dependency outputs.

JavaScript/JSX, Liquid, CSS, Prisma, TOML; npm lockfile and pnpm workspace. Node >=20.19 <22 or >=22.12. Scripts include dev/build/start/docker-start/lint/typecheck/setup, app Prisma generate/migrate, worker:notify-restock, and focused test:* suites. No npm test script or scheduler/deploy workflow.

Manifest / lock versions: React Router packages 7.18.2/7.18.2; Prisma and @prisma/client ^6.16.3/6.19.3; Shopify React Router ^1.2.1/1.2.1; session storage ^9.0.0/9.0.1; App Bridge ^4.2.4/4.2.13; React ^18.3.1/18.3.1; Vite ^6.3.6/6.4.3; TypeScript ^5.9.3/5.9.3; Polaris types ^1.0.1/1.1.0; ESLint ^8.57.1/8.57.1. Full direct inventory is in package.json/package-lock.json.

Git status showed project files untracked; history baseline unavailable. .env exists and is ignored. Names declared (values withheld): APP_DATABASE_URL, NOTIFY_CONTACT_HMAC_SECRET, SHOPIFY_API_KEY, SHOPIFY_API_SECRET, SCOPES. Presence does not verify valid values. Other referenced names include SHOPIFY_APP_URL, SHOP_CUSTOM_DOMAIN, APP_URL, HOST, PORT, FRONTEND_PORT.

4. Merchant pages and storefront

Merchant pages (all under Shopify admin auth): /app — Overview flags/links; /app/reporting — summary metrics, 7/30/custom 1–90 day range; /app/reporting/demand — product search, status, paging, waiting contacts, clicks, signups, cycle and failed-job retry; /app/reporting/wishlist — saves, episodes, matched products, active savers/ranking; /app/settings — GET/PATCH feature flags and supported surfaces. Sources: app/routes/app._index.jsx, app.reporting.jsx, app.reporting_.demand.jsx, app.reporting_.wishlist.jsx, app.settings.jsx. Loading/error/retry and empty states are present.

Theme blocks target product sections only. notify-me.liquid provides unavailable-variant dialog, email, required consent, disabled WhatsApp and theme-editor copy/color. notify-me.js tracks selected variant and records click. wishlist-save.liquid provides save/login control. wishlist-save.js stores one guest pending item and resumes after login. storefront-settings.js hides blocks unless settings fetch succeeds and flag is true. No collection-card UI exists.

My Wishlist: /apps/konvert-notify/my-wishlist. Uses signed customer identity and live Admin GraphQL data. Add-to-cart only for available saved variant; unavailable/invalid/deleted products remain removable. Guest localStorage key konvert-wishlist-pending TTL 30 minutes; sessionStorage konvert-wishlist-login-attempt; URL flags konvert_wishlist_resume and konvert_wishlist_saved. Return paths are same-origin sanitized. Theme compatibility was not tested.

5. API and webhook inventory

GET/POST /auth/login; GET /auth/* (Shopify login/auth). GET /app/* (merchant pages). GET/PATCH /api/settings (admin-auth feature settings; PATCH JSON <=4KiB). GET /api/notify/settings (signed proxy flags). POST /api/notify (signup; JSON <=16KiB). POST /api/notify/analytics (click; <=4KiB). GET/POST/DELETE /api/notify/wishlist (signed customer; <=8KiB). GET /api/notify/my-wishlist (signed

proxy/customer page). GET /api/reporting and /api/reporting/wishlist (admin reports, days 1–90). GET/POST /api/reporting/demand (report/retry); GET /api/reporting/demand/contacts (operational contact data). GET /api/check-scopes (admin). POST /webhooks/orders/paid; POST /webhooks/app/uninstalled; POST /webhooks/app/scopes_update (Shopify webhook auth).

Proxy mapping in shopify.app.toml: /apps/konvert-notify -> /api/notify. JSON routes use no-store. Input bounds/allowlists exist on key routes; no app-level rate limiter found. No explicit Origin/CSRF check found; framework protection NOT VERIFIED. Contacts API intentionally returns normalized contact to authenticated merchant.

6. Database and Prisma data dictionary

prisma/schema.prisma: SQLite Session storage wired via app/db.server.js and PrismaSessionStorage. prisma/application/schema.prisma: MySQL via APP_DATABASE_URL, generated to app/generated/application-client, used by app/application-db.server.js. Actual live schema/migrations NOT VERIFIED. MySQL DATETIME(3) has no timezone; code uses JS Date.

Shop (baseline table shop): id cuid PK; shopifyShopId String(128) unique; domain String(255) unique; notifyEnabled/wishlistEnabled Boolean default true; createdAt now, updatedAt @updatedAt.

NotifyRequest (notifyrequest): id cuid; shopId; product/variant Shopify IDs String(128); customer GID String(128)?; normalizedContact String(320) raw normalized contact; contactDeduplicationKey String(128) HMAC; channel enum; consentText Text; consentVersion String(64); consentedAt; source enum; status default PENDING; createdAt/updatedAt; closedAt?. Shop FK cascade. Indexes shop/variant/digest/status, shop/product/variant, customer/status, status/time; no unique open-request constraint.

WishlistEntry (wishlistentry): id cuid; shop/customer/product IDs; preferredVariant ID?; savedAt default now; isActive true; deactivatedAt?; createdAt/updatedAt. Unique shop/customer/product; indexed customer/active/time and product/active. One reusable row; event history separate.

VariantAvailability (variantavailability): id cuid; shop/product/variant IDs; previousAvailable Boolean?; latestAvailable Boolean; lastCheckedAt; restockCycle Int default 0; createdAt/updatedAt. Unique shop/variant; product and availability indexes.

HandoffJob (handoffjob): id cuid; notifyRequestId; idempotencyKey String(191); status enum default PENDING; attempts 0; retryAt?; providerReference String(191)?; sanitizedErrorCode String(100)?; sanitizedErrorMessage String(500)?; createdAt/updatedAt. FK request cascade; unique request/key; indexes status/retryAt and request/time.

AnalyticsEvent (baseline analyticsevent): id cuid; shopId; eventKey String(191); type enum; occurredAt now; product/variant/customer IDs?; email/phone digest String(64)?; notifyRequestId?/wishlistEntryId?; orderShopifyId String(128)?; sourceSurface String(64)?; restockCycle Int?; createdAt now. Unique shop/eventKey; shop FK cascade, request/Wishlist SET NULL; indexes by event/time, product, customer, surface, variant/cycle, order and entity/time.

PaidOrderEvent (PaidOrderEvent): id cuid; shopId; webhook ID String(191); order GID String(128); orderCreatedAt; paidAt?; customer GID?; email/phone digest String(64)?; receivedAt/createdAt now. Unique shop/webhook and shop/order; identity/time indexes. PaidOrderLine (PaidOrderLine): id cuid; parent ID;

order-line GID String(128); product/variant GID?; unique parent/line; product/variant indexes; parent cascade.

OrderMatch (baseline ordermatch): id cuid; shop/order IDs; order-line ID?; PaidOrderLine ID?; source enum; request/Wishlist IDs?; orderCreatedAt; paidAt?; matchedAt now; orderTotal Decimal(12,2)?; currencyCode String(3)?; createdAt/updatedAt. Unique shop/order/line/source and paidLine/source; indexed shop/time and entity/time. Shop cascade; other FKs SET NULL. Matcher does not populate revenue fields.

SQLite Session: id String PK, shop/state String, isOnline false, scope/expires?, accessToken, userId BigInt?, names/email?, accountOwner false, locale?, collaborator/emailVerified Boolean? default false, refreshToken?/expiry?. Contains OAuth token fields.

Enums: NotifyChannel EMAIL/SMS; NotifyRequestSource THEME_APP_EXTENSION/STOREFRONT/ADMIN/API/IMPORT; NotifyRequestStatus PENDING/ACTIVE/NOTIFIED/CLOSED/UNSUBSCRIBED; HandoffJobStatus PENDING/PROCESSING/SUCCEEDED/FAILED; AnalyticsEventType NOTIFY_REQUEST_CREATED/NOTIFY_SENT/NOTIFY_CLICKED /WISHLIST_CLICKED/RESTOCK_DETECTED/HANDOFF_ACCEPTED/HANDOFF_FAILED/WISHLIST_ENTRY_CREATED/WISHLIST_EN REMOVED/PRODUCT_VIEWED/ORDER_PLACED; OrderMatchSource NOTIFY_REQUEST/WISHLIST_ENTRY.

Migration risk: baseline uses lowercase core table names but later migrations refer to AnalyticsEvent/Shop/PaidOrderEvent; no Prisma @@map declarations. Case-sensitive MySQL compatibility NOT VERIFIED. Migration filenames 20261019–20261024 are after audit date 2026-10-12.

7. ER diagram and relationships

```
erDiagram
Shop ||--o{ NotifyRequest : owns
Shop ||--o{ WishlistEntry : owns
Shop ||--o{ VariantAvailability : observes
Shop ||--o{ AnalyticsEvent : records
Shop ||--o{ PaidOrderEvent : ingests
Shop ||--o{ OrderMatch : scopes
NotifyRequest ||--o{ HandoffJob : queues
PaidOrderEvent ||--|{ PaidOrderLine : contains
PaidOrderLine ||--o{ OrderMatch : matched_by
WishlistEntry ||--o{ AnalyticsEvent : referenced_by
```

Foreign keys do not by themselves enforce same-shop linkage. Shop deletion cascades app data; request deletion cascades jobs; event/match request/Wishlist refs become null.

8. Database write/read lifecycle matrix

Merchant auth: Admin GraphQL shop identity; upsert Shop. Notify: signed proxy, verified variant, feature flag and open request; create request + NOTIFY_REQUEST_CREATED transactionally or duplicate response. Wishlist: signed customer/product verification; row create/reactivate/update/deactivate plus episode/click events. Availability: pending variants and GraphQL; update current row, transition event and unique cycle jobs. Handoff: claim/status/retry writes and limited outcome events. Paid order: event/lines/ORDER_PLACED transaction. Matching: paid lines plus event/digest eligibility; unique OrderMatch.

9. Notify Me lifecycle

Browser validates email and consent and posts variant/contact/EMAIL/consent via App Proxy. Server validates JSON type/size, email syntax, signed shop/session, optional customer ID and variant/shop via Admin GraphQL. Email is trimmed/lowercased; NOTIFY_CONTACT_HMAC_SECRET HMAC-SHA256 with EMAIL prefix requires >=32 bytes. Transaction locks Shop, deduplicates same shop/variant/digest in PENDING or ACTIVE. Duplicate: HTTP 200; create request/event: HTTP 201. Consent text/version/time are stored. Server does not check availability at submit; a newly available variant may still be registered.

10. Wishlist lifecycle

Uses signed proxy customer identity; browser customer_id is rejected. Product/variant verified by Admin GraphQL; contact digest lookup best-effort. Unique shop/customer/product row is reused: active save updates preference, remove deactivates, re-add reactivates/resets savedAt. Episode history is AnalyticsEvent, not separate rows. Login resume stores one guest item for 30 minutes; cancel clears it. Cross-device persistence requires login. Deleted/invalid product remains removable; collection controls absent.

11. Availability and handoff lifecycle

Worker polls pending-request variants via Admin GraphQL availableForSale. Initial observation previousAvailable=null/cycle 0; later false->true increments cycle and records RESTOCK_DETECTED. Restock processing confirms transition and creates one request/cycle job. No availability webhook or missed-signal recovery. Handoff states PENDING->PROCESSING->SUCCEEDED or PENDING with retryAt then FAILED; max 3 attempts with 60s/120s delays. Manual retry resets attempts/errors, retains idempotency key. Accepted means tool acceptance, not delivery.

processPendingHandoffJobs exists but no runtime caller was found. app/workers/notify-restock.js runs availability and purchase matching only. No scheduler is encoded; retry action merely queues a job.

12. Marketing provider status

provider-adapter.js has configured=false and returns permanent provider_not_configured. No concrete provider, provider credentials, delivery callback, opt-out/suppression or config UI found. Adapter payload would contain raw contact, channel, consent, Shopify IDs and idempotency key. Error fields are sanitized/bounded. No real send was tested.

13. Analytics event catalogue

NOTIFY_CLICKED: browser modal open, verified by server, click UUID (not unique sessions). NOTIFY_REQUEST_CREATED: transactional signup, stable request key. RESTOCK_DETECTED: observed false->true, variant/cycle key. HANDOFF_ACCEPTED: adapter accepted, job key, not sent. HANDOFF_FAILED: some terminal failures only. WISHLIST_CLICKED: add/update/remove, random key. WISHLIST_ENTRY_CREATED/REMOVED: episode start/end, random keys. ORDER_PLACED: verified paid-order event. NOTIFY_SENT and PRODUCT_VIEWED are enum-only; producers not found.

PRD names differ: uppercase enums; signup is NOTIFY_REQUEST_CREATED. No order_matched, signup_failed, login_resume_failed or integration_error events. Match is stored as OrderMatch, not analytics. Raw

contacts are in NotifyRequest and authenticated contacts response, not event columns; events can contain Shopify customer IDs and HMAC digests.

14. Paid order ingestion and purchase matching

orders/paid uses authenticate.webhook. Only financial_status=paid and valid order/line IDs and order creation date ingest. Stores IDs/times/customer GID and HMAC contact digests, no raw contact. Unique shop/webhook and shop/order prevent duplicates. Missing triggeredAt yields paidAt=null; matcher requires paidAt. Handler returns 200 after ingestion call regardless of result status.

Notify match: exact variant, HANDOFF_ACCEPTED required, strict anchor < paidAt < anchor+7 days; identity prioritizes customer ID then channel digest, conflict checked, newest eligible anchor wins. Wishlist match: product-level; save precedes purchase, removal must not be at/before purchase, strict seven-day window; newest eligible episode selected. One match per line/source; line may have both sources. Worker retries unmatched lines. No refund, cancellation, quantity, line-price, revenue or currency processing. No match analytics event.

15. Reporting formulas and provenance

Window: rolling now-days*24h to now, start inclusive/end exclusive; API 1–90 days; UI 7/30/custom. Notify clicks = event rows (not sessions); signups = distinct identity on request-created events; signup rate = unique signups/click events. Waiting requests = PENDING rows created during window. Handoff attempts = attempted jobs updated in window; accepted/failed = event counts; acceptance rate = accepted/attempted, provider-gated. Matched buyers = distinct identity on OrderMatch paidAt in range; observed association, not causation.

Demand waiting contacts = distinct HMAC per variant; Demand clicks/signups are all-time by variant (no date filter). Wishlist saves = create episode rows; active savers = distinct customer IDs; item conversion = distinct cohort products later matched / distinct cohort products saved. Lifetime active Wishlist total and unique sessions are not supported. Zero denominator rates are null; provider values unavailable when unconfigured.

16. Security, privacy and compliance

Shopify admin/proxy/webhook authentication is used; shop scope generally enforced, but cross-entity same-shop consistency is not DB-enforced. Wishlist rejects browser customer IDs. Contact is raw in NotifyRequest and operational contacts output; HMAC digests used for matching. Body bounds/allowlists exist; no app rate limiting found. Explicit Origin/CSRF checks not found; framework protections NOT VERIFIED. No customer/shop redaction webhook or data retention worker; uninstall handler deletes sessions only. Consent is recorded but unsubscribe/suppression absent. Logs generally report generic names, not contact values.

17. Settings and toggles

Shop.notifyEnabled and wishlistEnabled default true. GET/PATCH /api/settings admin-authenticated; strict boolean feature object, JSON <=4KiB. Signed proxy settings feed blocks. Storefront fails closed (hidden on retrieval failure). APIs enforce feature flags; My Wishlist returns 404 if explicitly disabled. Existing data/jobs remain. Email enabled; WhatsApp disabled/coming soon; provider state not persisted

merchant setting.

18. Testing inventory

Node node:test suites mainly service-level/fake DB and source-text checks; no browser E2E or live MySQL integration suite identified. Tests not run; passing status NOT VERIFIED. Areas: Notify submission/click/theme JS; settings; Wishlist persistence/identity/client/page/remove; availability/restock/worker; handoff retry; order ingestion/matching; reporting summary/routes/UI/demand/Wishlist; home source checks. Focused npm run test:* scripts are listed in package.json. A1–A12 coverage NOT VERIFIED because PRD unavailable.

19. Deployment and operations

Docs describe npm ci, Shopify CLI linking, SQLite setup, APP_DATABASE_URL/HMAC config, app Prisma generate/migrate, npm run dev. MySQL setup is separate. Docker uses npm ci --omit=dev then docker-start->setup, but Prisma CLI is a devDependency and setup invokes it; production startup may fail. Setup targets SQLite and does not generate/apply MySQL client/schema. No scheduler, monitoring, backups or provider config encoded.

TOML API version 2027-01 vs server ApiVersion.July26; live validity NOT VERIFIED. TOML app/auth URLs are example.com placeholders; proxy URL is trycloudflare. Shopify Admin state not verified.

20. PRD compliance matrix

Embedded UI PARTIAL (implemented; deployment unknown). Notify PARTIAL (product signup; no collection or submit-time availability check). Wishlist PARTIAL (product save/login/page; one pending guest item). Provider delivery MISSING. Availability PARTIAL (polling code; no scheduler/webhook recovery). Handoff processing PARTIAL (service exists but not wired into worker). Paid-order ingestion PARTIAL (code exists; active setup unknown). Matching PARTIAL (service/worker exist; no refunds/match event). Reporting PARTIAL (implemented; provider unavailable and some metrics all-time). Feature toggles IMPLEMENTED in code; live state unknown. GDPR retention/deletion MISSING. V1/A1–A12 NOT VERIFIED.

21. Limitations and unverified facts

Not verified: live MySQL schema/data/migration state; Shopify installation/scopes/webhook/proxy/theme blocks; credentials validity; provider/scheduler; protected customer data; test results; PRD. Migration case mismatch needs review. README/architecture docs conflict with current reporting/matching code. Worker does not process handoff jobs; no retention/deletion workflow found.

22. Prioritized follow-up and safe checks

1. Verify MySQL migration/table casing and applied state. 2. Wire a scheduled HandoffJob processor and choose provider. 3. Confirm Shopify scopes, webhook, proxy and Theme Editor setup. 4. Define deletion/retention/unsubscribe/suppression. 5. Decide behavior when stock returns before signup completes. 6. Obtain V1 PRD, map tests and execute relevant local suites. 7. Reconcile docs and Docker startup with split databases.

Safest checks: read-only information_schema metadata and _prisma_migrations in Workbench (no

customer/contact queries); Shopify Admin inspect scopes/webhooks/proxy/theme blocks; inspect hosting secrets/scheduler without copying values. Shopify guidance used: https://shopify.dev/docs/apps/build/ai-toolkit.